

Business Analyst Generic Round - 1 Q n A's

1. What is Agile? How have you worked in Agile practically?

Agile is a mindset, not just a process. It means delivering value in small, iterative increments rather than waiting for everything to be perfect at the end.

How I've worked practically:

In my last project, we had 2-week sprints. Every sprint, we delivered 1-2 fully functional features not half-built things.

We had daily stand-ups where I shared requirement clarifications and blocked dependencies.

Every sprint planning, I walked in with groomed user stories that were 'ready' acceptance criteria written, dependencies mapped.

Mid-sprint, if a stakeholder asked for a change, I didn't say 'no' or 'yes' immediately. I said: 'Let me check impact and we'll decide in daily stand-up.'

After each sprint, I attended retro and shared one thing: 'What confused the dev team last sprint?' Then I fixed my documentation.

One example: We were building a payment integration. Instead of waiting 2 months for full API documentation, I worked with the vendor to get a mock endpoint in sprint 1. We built the UI first, then swapped real API in sprint 2. That's Agile delivering early, adjusting fast.

2. Difference between Agile and Waterfall with a real example?

The core difference is how you handle uncertainty.

Waterfall says: plan everything upfront, then build, then test, then launch. Works when requirements are fixed like building a bridge.

Agile says: build a little, get feedback, adjust, build more. Works when requirements evolve like building a new app nobody has used before.

Real example from my experience:

Waterfall example: I worked on a regulatory reporting project for a bank. The format was fixed by the government. No changes allowed. We used Waterfall BRD signed first, then development, then testing. Any change required a formal change request taking 2 weeks.

Agile example: Same company, different project a customer mobile app. Users kept asking for new features. We used Agile. In sprint 1, we built login + profile. Users said 'we want fingerprint login.' In sprint 2, we added it. You can't do that in Waterfall.

Simple rule I follow:

Requirements clear and fixed? → Waterfall

Requirements will change? → Agile

Don't know? → Start Agile, you can always add ceremony later.

3. How do you gather requirements in a project?

I use a 4-step approach not just one method.

Step 1 -- Discover the 'why' before the 'what'

I don't start with 'what screen do you want?' I ask: 'What problem are we solving? Who is the user? What does success look like?'

Step 2 -- Multiple techniques depending on situation

Situation	Technique
New project, unclear scope	User interviews + journey mapping
Existing system improvement	Process flow analysis + pain point workshop
Missing details	Prototyping + walkthrough
Technical integration	API documentation review + sequence diagrams

Step 3 -- Document in small chunks

I don't write a 100-page BRD anymore. I write user stories with acceptance criteria and get sign-off within 2-3 days, not weeks.

Step 4 -- Validate with the right people

I review with dev (is it buildable?) and QA (is it testable?) before showing the stakeholder.

Real example: On an e-commerce project, stakeholders said 'we want a return portal.' I asked 'why?' They said 'customers call support too much.' I mapped the current call flow, found 70% of calls were about 'where is my refund?' So instead of a portal, we added a refund tracking feature. Solved the problem with 1/3 the effort. That came from asking 'why' first.

4. What challenges did you face during requirement gathering?

Three challenges I've faced repeatedly:

Challenge 1 -- Silent stakeholders

Situation: One stakeholder never spoke in meetings. Later, during UAT, he rejected the feature. What I did: I started sending him a 5-minute 'pre-read' one day before meetings with 3 specific questions marked 'Need your input.' He started replying via email. Not ideal, but better than silence.

Challenge 2 -- Assumptions becoming requirements

Situation: Developer assumed 'invoice will be generated automatically.' Business assumed 'someone will review it manually.' Neither was documented. What I did: I now run a 'pre-refinement' session where I ask dev and QA separately: 'What do you assume here?' I document every assumption in the story.

Challenge 3 -- 'Just build what Amazon does'

Situation: Stakeholders give vague references instead of requirements. What I did: I ask them to spend 10 minutes using that reference and note 3 specific behaviors they want. Then we build only those, not the entire Amazon.

Key lesson: Requirement gathering is not about collecting. It's about uncovering what people forgot to say.

5. How do you prioritize requirements? (MoSCoW / RICE)

I use different methods for different situations.

MoSCoW -- When stakeholders can't agree and need a simple framework

Category	Meaning	Example
Must have	System fails without it	Login, payment
Should have	Important but workaround exists	Email notification
Could have	Nice to have, defer if needed	Dark mode
Won't have (this time)	Explicitly excluded	Social login

Real example: For an MVP launch, I forced stakeholders to limit 'Must have' to 60% of capacity. Everything else went to 'Should' or 'Could'. Painful for one day, but saved 3 months of delays.

RICE -- When I need data-driven prioritization for features

Reach -- how many users affected?

Impact -- how much does it move the metric?

Confidence -- how sure are we?

Effort -- developer days

Formula: $(\text{Reach} \times \text{Impact} \times \text{Confidence}) / \text{Effort} = \text{RICE score}$. Highest score wins.

My practical rule:

Early in project → MoSCoW (quick alignment)

After launch → RICE (optimize for value)

Emergency → just ask 'What breaks if we don't do this?'

6. Which SDLC model is best for your project and why?

I don't have a favorite. I choose based on three factors:

Factor 1 -- Requirement clarity

Clear & fixed → Waterfall or V-model

Unclear or changing → Agile (Scrum/Kanban)

Factor 2 -- Team size & location

Small, co-located team → Scrum works beautifully

Large, distributed team → SAFe or hybrid

Factor 3 -- Risk & compliance

Medical device / banking → V-model with heavy documentation

Internal tool → Kanban with lightweight process

Real example from my last project:

We built an API integration for a bank.

Compliance needed traceability → Waterfall for documentation

But API specs kept changing → Agile for development

What I did:

Requirements + sign-off = Waterfall (BRD, FRD, traceability matrix)

Development + testing = Agile (2-week sprints, daily stand-ups)

It's called Hybrid and it worked.

My answer to 'which is best?' : 'The one that fits your project's uncertainty, team structure, and compliance needs. I can explain my recommendation in 5 minutes after understanding these three things.'

7. What is a User Story? How do you write it?

A user story is a small, valuable piece of functionality written from the end user's perspective. It's not a requirement document it's a conversation starter.

Standard format (INVEST principle):

As a [type of user], I want [some goal] so that [some reason].

Example -- Bad vs Good:

Bad

'Build a search bar'

Good

'As a customer, I want to search products by name so that I can find items faster without browsing categories.'

How I write stories in real projects:

Title -- short (max 8 words)

'Product search by name'

User story -- the standard format

'As a customer, I want to search products by name so that I can find items faster.'

Acceptance criteria -- Given/When/Then format

Given I am on the homepage, when I type 'laptop' and click search, then I see only products with 'laptop' in the name.

Technical notes -- for dev team

API endpoint: GET /products?search=laptop | Pagination: 20 results per page

Dependencies

Search index must be updated by EOD

My rule: A story is ready when a developer can estimate it and a QA can write test cases without asking me more than 2 clarifying questions.

8. What is acceptance criteria? Give an example.

Acceptance criteria (AC) are pass/fail conditions that define when a user story is done and working correctly. If AC passes, business accepts. If AC fails, story is not done.

I use Given-When-Then format because it's unambiguous and testable.

Example -- Payment feature:

Story: As a customer, I want to pay using a credit card so that I can complete my purchase.

Acceptance criteria:

#	Given	When	Then
1	I have items in cart	I enter valid card details and click 'Pay'	Order is created, payment success message shown
2	I enter invalid card number	I click 'Pay'	Error message: 'Invalid card number'
3	I have insufficient balance	I click 'Pay'	Error message: 'Insufficient funds'
4	Payment gateway times out	I wait 30 seconds	Error message: 'Payment failed, please try again'

Why this works:

Dev knows exactly what to build

QA writes test cases directly from AC

No debate about 'done'

My practical tip:

I write AC before refinement. Then in refinement, I ask the team: 'What scenario did I miss?' They always find 2-3 edge cases I didn't think of.

9. What is RTM and why is it important?

RTM stands for Requirements Traceability Matrix. It's a table that links requirements to test cases and sometimes to design or code.

Simple structure:

Requirement ID	Requirement Description	Test Case ID	Status
REQ-101	User can log in with email/password	TC-201, TC-202	Covered
REQ-102	User sees error after 3 failed attempts	TC-203	Covered
REQ-103	Password reset via email	Not yet	Missing

Why it's important in real projects:

No missed requirements -- If a requirement has no test case, it won't be tested = production bug.

Impact analysis -- When a requirement changes, RTM tells me exactly which test cases to update.

Audit & compliance -- In banking/healthcare, auditors ask: 'Prove every requirement was tested.' RTM is your proof.

How I use RTM practically:

I don't maintain it manually in Excel for large projects that breaks. Instead, I use Jira with 'Requirement' issue type linked to 'Test' issues. The traceability is automatic.

For small projects: A simple Google Sheet with 3 columns: Requirement → Test Case → Pass/Fail. Takes 10 minutes to create, saves 2 hours of 'did we test this?' arguments.

10. How do you handle requirement changes mid-project?

I follow a 4-step change management process not a 'no' or 'yes' immediately.

Step 1 -- Understand the change

'What problem does this solve? What happens if we don't do it now?'

Step 2 -- Assess impact

Change Size	Impact	Action
Small (≤ 2 hours)	Low	Team + PO agrees, update ticket
Medium (half day)	Medium	Swap equal work, adjust sprint
Large (1+ days)	High	Defer to next sprint

Step 3 -- Check sprint goal

'If we add this, does the sprint goal still make sense?'

If yes → negotiate swap

If no → defer or reprioritize sprint

Step 4 -- Communicate & document

Update ticket with change log. Inform QA (regression risk). Update estimates.

Real example:

Mid-sprint, PO asked to add a 'referral code' field. Impact = 4 hours (medium). I checked with dev they could swap a low-priority bug fix. Got team approval. Updated the story. QA added 2 test cases. Delivered on time.

What I never do:

Say 'yes' without impact analysis. Say 'no' without offering alternatives.

11. How do you ensure requirements are clear for developers?

I learned this the hard way after a developer built the wrong feature twice.

My 3-pillar approach:

Pillar 1 -- Write for clarity, not completeness

No paragraphs. Use bullet points, tables, examples.

Avoid 'the system shall' use 'When X, then Y'.

Add a 'What this story does NOT cover' section.

Pillar 2 -- Walk through, don't throw over the wall

Before refinement: I spend 15 minutes with a senior dev walking through the story. They ask 'What about this edge case?' I add it to AC.

Before sprint: I do the same with QA.

Pillar 3 -- Use visuals

Sequence diagram for API flows

State machine for status changes

Mock screenshots for UI

One technique that changed everything for me:

The 2-question rule After I write a story, I ask a dev: 'What two things are still unclear?' If they can name 2, I rewrite. If they can't, the story is ready.

Example of unclear vs clear:

Unclear

'User can filter products'

Clear

'Filter by price range (slider: ₹0-₹10,000), brand (dropdown), rating (1-5 stars). Default: no filter.'

My goal: A developer should not need to interrupt me during the sprint to ask basic questions.

12. What is your role in UAT?

UAT (User Acceptance Testing) is when the business users test the system in their real-world scenarios. My role is not to test it's to enable them to test successfully.

My 5 responsibilities in UAT:

#	Role	What I actually do
---	------	--------------------

1	Gatekeeper	Verify that system testing is complete before UAT starts. No known critical bugs.
2	Preparer	Create UAT test cases based on business scenarios, not technical edge cases.
3	Coordinator	Schedule users, provide credentials, set up test data.
4	Clarifier	When users ask 'Is this a bug or expected?', I check requirements and answer.
5	Sign-off facilitator	Collect UAT sign-off email/approval. No sign-off = no go-live.

Real example:

On an ERP migration project, users failed UAT because they couldn't find the 'approve' button. It wasn't a bug the button was renamed to 'authorize' in requirements. But I missed communicating that change. I learned: Send a 'What changed since last time?' document before UAT starts.

What I am NOT responsible for in UAT:

Fixing bugs (that's dev). Writing automation scripts (that's QA). Making business decisions (that's PO). I enable, communicate, and document.

13. Explain one project end-to-end in 2 minutes.

Answer (use a real project structure -- this is a template you can replace with your own):

Let me walk you through an e-commerce order tracking project I worked on.

Problem (15 seconds)

Customers were calling support 500 times a day asking 'Where is my order?' Support team was overloaded.

My role (10 seconds)

I was the sole BA responsible for requirements, stakeholder alignment, and UAT coordination.

Process -- Step by step (60 seconds)

Discovery (2 weeks) -- I interviewed 10 support agents, mapped the current 'call flow', and found 70% of calls were about refund status, not delivery.

Requirements (2 weeks) -- I wrote 15 user stories including 'refund tracking status' and 'order timeline view'. Used MoSCoW to prioritize.

Development (6 weeks) -- 3 sprints. Midway, payment team changed their API. I managed the change by deferring a low-priority feature.

Testing (2 weeks) -- Worked with QA on 50 test cases. Supported UAT with 5 business users.

Go-live & results (15 seconds) -- Support calls dropped by 60%. Customer NPS improved by 12 points.

What I learned (10 seconds)

I learned that asking 'why' first saved us from building a full order tracking portal when a simple refund status view solved 70% of the problem.

That's the project in 2 minutes. I can go deeper on any phase if you'd like.

Business Analyst Generic Round - 1 Q n A's by Harish Kavire